Formally Verified Code Generation via Petri Nets for Rust's Ownership and Lifetimes

Kaiwen Zhang
Guanjun Liu*
zhangkw@tongji.edu.cn
liuguanjun@tongji.edu.cn
Tongji University
Shanghia, Shanghai, China

Abstract

Rust libraries increasingly expose stateful, type-directed APIs whose effective testing requires generating long, well-typed call sequences rather than mutating unstructured byte inputs. However, Rust's affine ownership, borrow exclusivity, and lifetime constraints impose a compilation barrier: naively generated sequences frequently fail to type-check due to move-after-use, illegal aliasing under &mut, or unsatisfied generic bounds, preventing any runtime feedback. This paper addresses signature-level test generation for Rust crates by modeling API invocations as a resource-flow problem over typed values and permissions.

We propose a Pushdown Colored Petri Net that is constructed from a crate's extracted function signatures. Places denote Rust type expressions and related compile-time obligations (including generic placeholders, trait satisfaction, associated types, and field projections), while transitions capture signature-driven function calls and a small set of structural transformations (e.g., borrowing, borrow termination, projection and re-borrowing of fields). Token flow represents ownership transfer and borrow-induced capability changes, and a pushdown stack enforces properly nested borrow extents, binding produced references to their provenance and restoring permissions when borrows end. To support generic-heavy APIs, we define a partial-order-based satisfaction relation that checks whether a concrete instantiation meets trait bounds and associated-type constraints during transition enabling. Composite types are handled via formal projection and (re)borrow transitions that preserve Rust's aliasing restrictions at the level of generated harness code. On top of the model, we formulate generation as a reachability problem with explicit coverage objectives over transitions (and, optionally, over signature items) and develop a bounded reachability analysis that searches for high-coverage firing traces under guard constraints.

Each trace is deterministically translated into a Rust harness, inserting drop or scope boundaries to realize stack-pop events, yielding programs that are compilable under the model's conservative borrowing discipline. We implement a prototype tool that integrates with the Rust toolchain for signature extraction and produces harnesses compatible with standard coverage-guided fuzzing workflows.

CCS Concepts

• **Do Not Use This Code → Generate the Correct Terms for Your Paper**; *Generate the Correct Terms for Your Paper*; Generate the Correct Terms for Your Paper; Generate the Correct Terms for Your Paper.

Keywords

Rust, Colored Petri Nets, Formal Modeling, Fuzzing, Type Systems, API Sequence Generation

ACM Reference Format:
Kaiwen Zhang and Guanjun Liu. 2018. Formally Verified Code Generation via Petri Nets for Rust's Ownership and Lifetimes. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX).* ACM, New York, NY, USA, 13 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Rust has become a mainstream choice for building reliable systems software and performance-critical libraries. Its combination of low-level performance and a rich type system eliminates broad classes of memory-safety bugs, while still exposing fine-grained control over resources. As a result, an increasing amount of security and correctness-critical functionality is delivered as reusable Rust crates. For such libraries, automated test generation especially coverage-guided fuzzing, is an indispensable technique for discovering panics, logic bugs, and unexpected corner cases that escape conventional unit tests.

Despite its success, effective test-case generation for Rust libraries remains substantially harder than for many other languages. Modern fuzzers excel at mutating byte arrays and driving a single entry function, but library APIs are often stateful and type-directed: coverage typically depends on constructing a web of interacting objects and invoking a long sequence

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, Woodstock, NY
© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

of API calls that respects resource protocols. In Rust, such protocols are enforced not only by runtime checks, but also by compile-time rules on ownership, borrowing, and lifetimes. Consequently, a large fraction of naïvely generated call sequences never reach execution: they fail to compile due to move errors, aliasing violations, or unsatisfied generic constraints. This compilation barrier turns into a coverage barrier.

A motivating example. Consider a simplified crate API in Listing 1, which resembles many real-world libraries (parsers, databases, protocol stacks, etc.). It features (i) composite types (Result, Option, Vec), (ii) lifetime-carrying handles that borrow from a parent object, and (iii) a generic entry point constrained by trait bounds.

```rust
pub struct Store { /* ... */ }
pub struct Tx<'s> { store: &'s mut Store /* ... */ }
pub trait KeyLike {
    fn as_bytes(&self) -> &[u8];
}

impl Store {
    pub fn open(path: &str) -> Result<Self, ()> { /* ... */ }
    // Borrowing & lifetime: a transaction borrows the store mutably.
    pub fn begin<'s>(&'s mut self) -> Tx<'s> { /* ... */ }
}

impl<'s> Tx<'s> {
    pub fn put<K: KeyLike>(&mut self, k: K, v: Vec<u8>) { /* ... */ }
    // Immutable borrow from &self, returns a reference tied to 's.
    pub fn get<'a, K: KeyLike>(&'a self, k: K) -> Option<&'a [u8]> { /* ... */ }
    // Ownership consumption: committing consumes the Tx value.
    pub fn commit(self) -> Result<(), ()> { /* ... */ }
}
```

Listing 1: A small Rust API exhibiting ownership transfer, borrowing, lifetimes, and generics.

A test generator that ignores Rust's ownership discipline might attempt to interleave calls in ways that are invalid in Rust. For instance, after creating a transaction tx that holds &mut store, a generator may try to call store.begin() again, or mutate store directly, while tx is still live—both are rejected by the borrow checker. Similarly, commit(self) consumes tx, so any subsequent use of tx triggers a move error. Finally, generic parameters such as K: KeyLike require the generator to produce a concrete type that satisfies the bound, rather than an arbitrary byte blob. These issues are not merely syntactic. In Rust, well-typedness encodes a large portion of the API protocol. Therefore, achieving high coverage for Rust libraries hinges on the ability to generate compilable and type-correct call sequences that respect ownership, borrowing, lifetimes, and generic constraints.

A wide range of automated testing techniques exist, including coverage-guided fuzzing, property-based testing, grammar-based generation, symbolic execution, and model-based testing. However, when applied to Rust library APIs, they commonly face at least one of the following limitations:

(1) **Byte-oriented inputs without typed object construction.** Many fuzzing setups operate on byte arrays and rely on a hand-written harness to translate bytes into structured objects. For stateful APIs, the harness must additionally manage long call sequences, intermediate values, and resource protocols. This manual effort is error-prone and often becomes the dominant bottleneck.
(2) **Limited support for Rust-specific resource constraints.** Generators that treat values as freely copyable struggle with Rust's affine ownership: values can be moved, partially moved, borrowed immutably, or borrowed mutably, with strict aliasing rules. Moreover, Rust lifetimes are typically inferred (e.g., non-lexical lifetimes), which means that the precise end of a borrow is not explicitly written in the source code—yet it is essential for deciding when subsequent mutations become legal.
(3) **Weak handling of composite and generic APIs.** Real crates expose APIs over nested algebraic data types, slices, smart pointers, and rich trait-bound generics. Without a principled mechanism for instantiating generics (and ensuring that candidate types satisfy trait bounds), generators either (i) avoid such functions, or (ii) produce ill-typed code that never compiles.
(4) **Poor scalability when internal state is over-modeled.** Some analysis-driven approaches attempt to model the state of the internal program to drive generation. Although powerful, these approaches can be expensive and brittle for large crates and are often unnecessary when the goal is the generation of API-level tests rather than the verification of the full program.

A parallel line of work explores Petri nets (and related automata) for generating event sequences and testing protocols. Classic Petri-net encodings, however, are typically *untyped* or only weakly typed: tokens represent abstract resources, but not Rust values with ownership and borrow capabilities. As a result, straightforward Petri-net-driven sequence generation tends to violate Rust's compile-time rules in three recurring ways: (i) it fails to enforce *affinity* (a moved token must not be reused), (ii) it cannot express *borrow exclusivity* (no mutable aliasing, no mutation under an outstanding immutable borrow), and (iii) it lacks a notion of *scope- and lifetime-bounded availability* (references must not outlive the borrowed owner, and borrow permissions must eventually return). Furthermore, generic constraints require a solver-like step (selecting admissible instantiations), which traditional Petri-net models do not provide.

Our perspective: signature-driven generation without heavy internal modeling. This paper targets a common and practically important setting: we are given a Rust codebase and we aim to generate test harnesses that invoke its public (or selected) functions and methods. Crucially, we assume that function signatures including input/output types, reference mutability, lifetimes, and generic bounds are available from the cargo doc. We do not attempt to precisely model the internal control flow or memory state of function bodies. Instead, we treat each function as an atomic transition whose type signature specifies how values (and their capabilities) can flow across call boundaries. This signature-level stance makes the approach lightweight and scalable, while still capturing the constraints that dominate compilation success for generated tests.

Approach overview: Pushdown Colored Petri Nets for Rust API fuzzing. We introduce a new model for Rust test-case generation based on *Pushdown Colored Petri Nets* (PCPNs). At a high level:

- Places represent pools of available *typed values* together with *capabilities* describing how they may be used (owned, immutably borrowed, mutably borrowed).
- Transitions represent *function calls* extracted from the target crate's signatures. A transition consumes tokens matching the function's input types/capabilities and produces tokens matching its output types/capabilities.
- Colors encode Rust types (including composites) and capability metadata (e.g., owned T, &T, &mut T).
- A pushdown stack models *nested scopes* and *borrow extents*. When a reference is created, the model pushes the borrowed provenance; when the reference becomes dead (e.g., explicitly drop'd or out of scope), the model pops the corresponding frame and returns permissions to the original owner.

This pushdown structure is essential for Rust-style lifetime reasoning: borrow relationships can nest (reborrowing), and long API sequences may create and discharge borrows repeatedly. A finite-state abstraction often either becomes overly conservative (blocking legal programs) or overly permissive (generating ill-typed programs). PCPNs provide a compact, compositional way to represent potentially unbounded, properly nested borrow patterns.

Generics via a satisfiability partial order. To support generic APIs, we equip the model with a *type satisfaction* relation written as a partial order $\preceq$. Intuitively, $A \preceq \mathcal{R}$ means that the concrete type A satisfies the generic requirement $\mathcal{R}$ (e.g., a set of trait bounds, lifetime constraints, and where-clauses). During transition firing, generic parameters are instantiated by selecting concrete types that are minimal (or otherwise prioritized) under $\preceq$ while remaining consistent with the surrounding token colors. This design cleanly separates: (i) the Petri-net execution semantics for sequencing, and (ii) the constraint reasoning needed to produce admissible generic instantiations. In practice, the partial order enables both sound pruning (reject impossible instantiations) and guided exploration (prefer simpler types early, diversify later).

From model to executable fuzz harnesses. From a PCPN execution trace, we synthesize a Rust harness that composes API calls and feeds the resulting program to a standard coverage-guided fuzzer. Listing 2 illustrates a typical generated skeleton for the API in Listing 1. The harness is compilable by construction: every use respects move semantics, borrow exclusivity, and lifetime boundaries.

```rust
fn harness(bytes: &[u8]) {
    // Construct base values (from bytes or small generators).
    let path = "tmp.db"; // tmp.db from Constant
    let mut store = match Store::open(path) { Ok(s) => s, Err(_) => return };
    // Begin a transaction: tx borrows store mutably.
    let mut tx = store.begin();
    // Choose a concrete key type K that satisfies K: KeyLike.
    // Here we pick Vec<u8> (assume the crate provides impl KeyLike for Vec<u8>).
    let k: Vec<u8> = bytes.to_vec();
    // Use tx while it is live; store cannot be used mutably here.
    tx.put(k.clone(), bytes.to_vec());
    let _v = tx.get(k);
    // Explicitly end the borrow extent and consume tx.
    let _ = tx.commit();
    // After commit, permissions return: store is usable again.
    let _tx2 = store.begin();
}
```

Listing 2: A compilable, signature-driven harness sketch generated from a PCPN trace.

Importantly, the generator does not need to model what Store::open, Tx::put, or Tx::get do internally. The signature alone determines how ownership and borrow capabilities move across call boundaries, which is exactly the information required to produce executable tests.

Contributions. This paper makes the following contributions:

(1) We formulate **signature-driven Rust test generation** as a typed resource-flow problem and propose **Pushdown Colored Petri Nets (PCPNs)** as a unifying model for generating compilable API call sequences under Rust ownership, borrowing, and lifetime constraints.
(2) We design a **capability-and-lifetime encoding** for Rust values in PCPNs, including composite types and reborrowing patterns, and show how pushdown stack discipline captures nested borrow extents with returned permissions.

(3) We introduce a **partial-order-based mechanism** for **generic instantiation** that checks whether concrete types satisfy trait-bound requirements, enabling exploration of generic-heavy APIs without manual stubs.
(4) We evaluate the method on a diverse set of real-world crates and demonstrate that PCPN-driven generation substantially increases the fraction of *compilable* generated tests and improves coverage for stateful APIs that are challenging for byte-oriented harnesses.

Section 2 reviews Rust features that matter for test generation and distills the key constraints imposed by ownership, borrowing, lifetimes, and generics. Section **??** presents the PCPN model and its capability semantics. Section **??** describes how we extract signatures, construct PCPNs, and synthesize compilable harnesses. Section 5 evaluates the approach on real-world crates and coverage-guided fuzzing workflows. Section **??** discusses related work in fuzzing, model-based testing, and Petri-net-driven sequence generation. Section **??** concludes and outlines future directions, including richer trait reasoning, environment modeling, and feedback-directed net refinement.

2 Background

2.1 Rust Ownership, Borrowing, and Lifetimes

Rust enforces memory safety primarily through a static discipline of ownership and borrowing. Every non-Copy value has exactly one owner at a time; transferring a value to another variable typically moves ownership. When the owner goes out of scope, the value is dropped and its resources are released.

Borrowing allows temporary access to a value without transferring ownership. Rust distinguishes: (i) shared borrows `&T`, which permit read-only access and can be aliased arbitrarily, and (ii) mutable borrows `&mut T`, which permit mutation but must be exclusive. The central rule is often summarized as *aliasing XOR mutability*: at any point, either many shared borrows exist, or exactly one mutable borrow exists.

A lifetime is a compile-time region that bounds how long a reference may be used. A reference cannot outlive the value it refers to, and borrow permissions return to the owner when the borrow ends. In modern Rust, borrow extents are often inferred (e.g., via non-lexical lifetimes), meaning the precise end point of a borrow is not necessarily obvious from surface syntax, but is still crucial for deciding whether later mutations are legal.

For test-case generation, these rules impose constraints on call sequences: a generator must track whether a value is still owned, has been moved, is currently shared-borrowed, or is mutably borrowed, and it must ensure that references do not outlive their provenance.

2.2 Generics and Trait Bounds in Signatures

Rust APIs are frequently generic. A function may quantify over type parameters and constrain them with trait bounds possibly in `where` clauses. For example, a signature `fn f<T: Trait>(x: T) -> U` restricts admissible instantiations of `T` to types implementing `Trait`. From the perspective of automated test generation, this means that producing a call is a two-step task: (1) choose concrete instantiations for generic parameters, and (2) supply values whose types match the instantiated signature. If step (1) fails, the generated program will not type-check, regardless of runtime behavior.

3 Method: Pushdown Colored Petri Nets for Rust API Test Generation

3.1 Problem Setting

Let $\mathcal{C}$ be a Rust crate. We can extract, from the compiler front-end, a finite set of *callable items* $\mathcal{F}$ (free functions and methods) together with their signatures, including (i) input/output types, (ii) reference mutability, (iii) explicit lifetime parameters and outlives constraints when present, and (iv) generic parameters with trait bounds and associated-type constraints.

Our goal is to generate compilable Rust *harnesses* that construct typed values and execute sequences of calls to items in $\mathcal{F}$. We deliberately avoid modeling internal states of function bodies; instead, each call is treated as an atomic transformation whose compile-time validity is determined by the signature-level ownership/borrowing/lifetime/generic constraints.

3.2 Petri Net Preliminaries

Definition 1 (Net and Petri Net). *A net is a 3-tuple $N = (P, T, F)$, where P is a finite set of places, T is a finite set of transitions, $F \subseteq (P \times T) \cup (T \times P)$ is a finite set of arcs, and $P \cap T = \emptyset$. A net can be viewed as a directed bipartite graph where places are represented by circles and transitions by rectangles. The pre-set and post-set of a node $x \in \{P \cup T\}$ are defined as ${}^\bullet x = \{y \in P \cup T \mid (y, x) \in F\}$ and $x^\bullet = \{y \in P \cup T \mid (x, y) \in F\}$, respectively. In this paper, it is assumed that the weight of each directed arc is 1.*

A marking of $N = (P, T, F)$ is a mapping $M : P \to \mathbb{N}$ where $M(p)$ is the number of tokens in place p. A net N with an initial marking M_0 is called a Petri Net and denoted as (N, M_0). Transition t is enabled at M if $\forall p \in {}^\bullet t : M(p) > 0$, denoted as $M[t\rangle$. A marking M is a deadlock marking if any transition is disabled at M. Firing an enabled transition t yields a new marking M': $M'(p) = M(p) - 1$ if $p \in {}^\bullet t \backslash t^\bullet$; $M'(p) = M(p) + 1$ if $p \in t^\bullet \backslash {}^\bullet t$; and otherwise $M'(p) = M(p)$. This is denoted as $M[t\rangle M'$. A marking M_k is reachable from M if $M_k = M$ or there exists a non-empty transition sequence $\sigma = t_1 t_2 \ldots t_k$ such that $M[t_1\rangle M_1[t_2\rangle \ldots M_{k-1}[t_k\rangle M_k$ (abbreviated as $M[\sigma\rangle M_k$). All markings reachable from M_0 in a Petri net is written as $\mathcal{M}$. This paper only considers bounded Petri nets, i.e., $\exists k \in \mathbb{N}, \forall M \in \mathcal{M} : M(p) \leq k$.

3.3 Pushdown Colored Petri Nets (PCPN)

Type expressions. Let GTy be the set of monomorphic Rust types extracted from $\mathcal{C}$. Let TVar be a set of generic type variables and LVar a set of lifetime variables. Let $\Sigma(\mathcal{C})$ denote the extracted signature/type environment.

We use the following type-expression language:

$$\tau \in \mathsf{Ty} ::= g \mid \alpha \mid \tau_0\langle\tau_1,\dots,\tau_n\rangle \mid (\tau_1,\dots,\tau_n) \mid [\tau] \mid \mathsf{Ref}^q_\ell(\tau) \mid \mathsf{Assoc}(\tau,\mathsf{Tr},A) \mid \mathsf{Field}(\tau,f) \mid \mathsf{Impl}(\tau,\mathsf{Tr}),$$

where $g \in \mathsf{GTy}$, $\alpha \in \mathsf{TVar}$, $q \in \{\mathsf{shr},\mathsf{mut}\}$, $\ell \in \mathsf{LVar}$, Tr ranges over traits, A over associated type names, and f over fields/projection paths.

Let $\ell_\bullet \in \mathsf{LVar}$ be a reserved lifetime variable that does not appear in extracted signatures. All internally created references are typed with $\ell_\bullet$ and carry the actual region label via $\lambda(\ell_\bullet)$.

Capability modes and places. Let $\mathsf{Cap} = \{\mathsf{own},\mathsf{frz},\mathsf{blk},\mathsf{val}\}$. We define the place set:

$$P = \{\, p_{\kappa,\tau} \mid \kappa \in \mathsf{Cap},\ \tau \in \mathsf{Ty} \,\}.$$

Intuitively, $p_{\mathsf{own},\tau}$ stores owned values; $p_{\mathsf{frz},\tau}$ stores owned values frozen by shared borrows; $p_{\mathsf{blk},\tau}$ stores owned values blocked by a mutable borrow; $p_{\mathsf{val},\tau}$ stores non-owning values (references, witnesses, etc.).

Colors and colored markings. Let $\mathbb{V}$ be a countably infinite set of value identifiers (fresh Rust variable names), and $\mathbb{L}$ a countably infinite set of region labels. Let Θ be finite type substitutions $\theta : \mathsf{TVar} \rightharpoonup \mathsf{GTy}$, and Λ finite lifetime substitutions $\lambda : \mathsf{LVar} \rightharpoonup \mathbb{L}$.

A token color is

$$c = \langle v,\theta,\lambda\rangle \in \mathsf{Col} := \mathbb{V}\times\Theta\times\Lambda.$$

A colored marking is

$$M : P \to \mathbb{N}^{(\mathsf{Col})},$$

i.e., each place carries a finite multiset of colored tokens.

Pushdown stack. Let Γ be a stack alphabet:

$$\gamma ::= \mathsf{Freeze}(v) \mid \mathsf{Shr}(v_o,v_r,\ell) \mid \mathsf{Mut}(v_o,v_r,\ell),$$

where $\ell \in \mathbb{L}$ is a region label. A configuration is $\mathsf{Cfg} = \langle M,S\rangle$ where $S \in \Gamma^*$ (top at the left).

Weighted arcs (or parallel arcs). We model arc multiplicities by a weight function

$$W : (P\times T)\cup(T\times P) \to \mathbb{N}$$

with finite support, and let $F = \{(x,y) \mid W(x,y) > 0\}$. For a transition t, we write $W(p,t)$ and $W(t,p)$ for input/output multiplicities.

Certificates, guards, and firing semantics. A *choice* for t at marking M is a multiset-valued function

$$\chi : P \to \mathbb{N}^{(\mathsf{Col})} \quad \text{s.t.} \quad \forall p \in P :\ \chi(p) \subseteq M(p) \wedge |\chi(p)| = W(p,t).$$

Let ι range over instantiations, containing at least:

$$\iota = (\theta^*,\lambda^*,\nu,\mu,\dots)$$

where $\theta^* \in \Theta$ and $\lambda^* \in \Lambda$ are the unified substitutions, and ν and μ provide any fresh value ids / fresh region labels needed by Out. Other resolved information such as associated-type resolution may also be stored.

Each transition t carries:

$$\mathsf{Guard}(t)(\langle M,S\rangle,\chi,\iota),$$
$$\mathsf{Out}(t)(\langle M,S\rangle,\chi,\iota) : P \to \mathbb{N}^{(\mathsf{Col})},$$
$$\mathsf{Act}(t) \in \{\epsilon, \mathsf{push}(\bar\gamma), \mathsf{pop}(\bar\gamma)\}.$$

Definition 2 (Enabledness and firing). *Let* $\mathsf{Cfg} = \langle M,S\rangle$. *A transition t is enabled at* Cfg *if there exist χ and ι such that* $\mathsf{Guard}(t)(\mathsf{Cfg},\chi,\iota)$ *holds and the pop-pattern (if any) matches the stack top. A firing rule is $\kappa = \langle t,\chi,\iota\rangle$.*

Given κ, firing yields $\mathsf{Cfg}' = \langle M',S'\rangle$ *where:*

- *Consume:* $M^-(p) = M(p) \ominus \chi(p)$ *for all p;*
- *Produce: let* $\pi = \mathsf{Out}(t)(\mathsf{Cfg},\chi,\iota)$ *with* $|\pi(p)| = W(t,p)$, *then* $M' = M^- \oplus \pi$;
- *Stack: S' is obtained by applying* $\mathsf{Act}(t)$ *(push/pop/ϵ) to S.*

3.4 Constructing a Rust PCPN from $\mathcal{C}$

We construct a crate-specific PCPN

$$\mathcal{N}_{\mathcal{C}} = (P,T,W,\mathsf{Col},\Gamma,M_0,\mathsf{Guard},\mathsf{Out},\mathsf{Act}),$$

where M_0 seeds a finite set of constructors (literals, empty containers, etc.).

Copy predicate. We write $\mathsf{IsCopy}(\tau)$ iff $\Sigma(\mathcal{C}) \vDash \tau : \mathsf{Copy}$.

Signature-induced transitions. For each callable item $f \in \mathcal{F}$ with signature

$$\mathsf{sig}(f) = \langle \bar\alpha;\bar\ell;\ \tau_1,\dots,\tau_n \to \tau_o;\ \rho\rangle,$$

we add a transition t_f whose arc weights are determined by the parameter passing mode:

1. If parameter i is by-value τ_i, then $W(p_{\mathsf{own},\tau_i},t_f)$+=1. Moreover, if $\mathsf{IsCopy}(\tau_i)$, then $W(t_f,p_{\mathsf{own},\tau_i})$+=1 (self-loop, no move).
2. If parameter i is shared reference $\mathsf{Ref}^{\mathsf{shr}}_\ell(\tau)$, then $W(p_{\mathsf{val},\mathsf{Ref}^{\mathsf{shr}}_\ell(\tau)},t_f)$+=1 and also $W(t_f,p_{\mathsf{val},\mathsf{Ref}^{\mathsf{shr}}_\ell(\tau)})$+=1 since $\&T$ is Copy.
3. If parameter i is mutable reference $\mathsf{Ref}^{\mathsf{mut}}_\ell(\tau)$, then $W(p_{\mathsf{val},\mathsf{Ref}^{\mathsf{mut}}_\ell(\tau)},t_f)$+=1. No self-loop by default: *&mut T* is not Copy.

If $\tau_o \neq ()$, we add one output arc:

$$W(t_f,\ p_{\kappa_o,\tau_o}) = 1$$

where $\kappa_o = \mathsf{own}$ if the return is owned, and $\kappa_o = \mathsf{val}$ if it returns a reference.

The guard $\mathsf{Guard}(t_f)$ solves: (i) type/lifetime unification across arguments, (ii) trait/assoc constraints in ρ, and (iii) capability feasibility (e.g., no direct use of blocked owners). $\mathsf{Out}(t_f)$ allocates fresh ids for returned values and reproduces Copy-arguments on self-loop arcs. $\mathsf{Act}(t_f) = \epsilon$ unless the signature semantically begins/ends a borrow-carrying owned value.

3.5 Ownership and Copy as Token Flow

Let $\pi_V(\langle v, \theta, \lambda\rangle) = v$, $\pi_\Theta(\cdot) = \theta$, and $\pi_\Lambda(\cdot) = \lambda$. For a singleton multiset $\{c\}$, write $\mathsf{one}(\{c\}) = c$.

(1) Move for non-Copy owned values. For each type τ, add a transition Move_τ with:

$$W(p_{\mathsf{own},\tau}, \mathsf{Move}_\tau) = 1,$$

and no outputs. Define:

$$\begin{aligned}
\mathsf{Guard}(\mathsf{Move}_\tau)(\langle M, S\rangle, \chi, \iota) &\equiv \neg\mathsf{IsCopy}(\tau),\\
\mathsf{Out}(\mathsf{Move}_\tau)(\langle M, S\rangle, \chi, \iota) &= \emptyset,\\
\mathsf{Act}(\mathsf{Move}_\tau) &= \epsilon.
\end{aligned}$$

(2) By-value use for Copy owned values (self-loop). For each τ, add $\mathsf{CopyUse}_\tau$:

$$W(p_{\mathsf{own},\tau}, \mathsf{CopyUse}_\tau) = 1, \qquad W(\mathsf{CopyUse}_\tau, p_{\mathsf{own},\tau}) = 1.$$

Let $c = \mathsf{one}(\chi(p_{\mathsf{own},\tau}))$. Define:

$$\begin{aligned}
\mathsf{Guard}(\mathsf{CopyUse}_\tau)(\langle M, S\rangle, \chi, \iota) &\equiv \mathsf{IsCopy}(\tau),\\
\mathsf{Out}(\mathsf{CopyUse}_\tau)(\langle M, S\rangle, \chi, \iota) &= [\, p_{\mathsf{own},\tau} \mapsto \{c\}\,],\\
\mathsf{Act}(\mathsf{CopyUse}_\tau) &= \epsilon.
\end{aligned}$$

Copy assignment to create a fresh variable id. Fix a per-type copy budget $B_{\mathsf{copy}}(\tau) \in \mathbb{N}$. Add $\mathsf{DupCopy}_\tau$:

$$W(p_{\mathsf{own},\tau}, \mathsf{DupCopy}_\tau) = 1, \qquad W(\mathsf{DupCopy}_\tau, p_{\mathsf{own},\tau}) = 2.$$

Let $c = \langle v, \theta, \lambda\rangle = \mathsf{one}(\chi(p_{\mathsf{own},\tau}))$ and let $v(\mathsf{new}) = v'$ be fresh. Define:

$$\begin{aligned}
\mathsf{Guard}(\mathsf{DupCopy}_\tau)(\langle M, S\rangle, \chi, \iota) &\equiv \mathsf{IsCopy}(\tau)\\
&\quad \wedge\ |M(p_{\mathsf{own},\tau})| < B_{\mathsf{copy}}(\tau)\\
&\quad \wedge\ v' \notin \mathsf{Ids}(M, S),\\
\mathsf{Out}(\mathsf{DupCopy}_\tau)(\langle M, S\rangle, \chi, \iota) &= [\, p_{\mathsf{own},\tau} \mapsto \{\langle v, \theta, \lambda\rangle,\ \langle v', \theta, \lambda\rangle\}\,],\\
\mathsf{Act}(\mathsf{DupCopy}_\tau) &= \epsilon.
\end{aligned}$$

(3) Drop. For each place $p_{\kappa,\tau}$, add $\mathsf{Drop}_{\kappa,\tau}$:

$$W(p_{\kappa,\tau}, \mathsf{Drop}_{\kappa,\tau}) = 1,$$

and no outputs. Define:

$$\begin{aligned}
\mathsf{Guard}(\mathsf{Drop}_{\kappa,\tau})(\langle M, S\rangle, \chi, \iota) &\equiv \top,\\
\mathsf{Out}(\mathsf{Drop}_{\kappa,\tau})(\langle M, S\rangle, \chi, \iota) &= \emptyset,\\
\mathsf{Act}(\mathsf{Drop}_{\kappa,\tau}) &= \epsilon.
\end{aligned}$$

(4) First shared borrow from an owned value. Add $\mathsf{BorrowShrOwn}_\tau$ as T_τ: $W(p_{\mathsf{own},\tau}, T_\tau)$=$W(T_\tau, p_{\mathsf{frz},\tau})$=$W(T_\tau, p_{\mathsf{val},\mathsf{Ref}^{\mathsf{shr}}_{\ell_\bullet}(\tau)})$=1.

Let $c_o = \langle v_o, \theta, \lambda\rangle = \mathsf{one}(\chi(p_{\mathsf{own},\tau}))$ and let $v(\mathsf{ref}) = v_r$, $\mu(\mathsf{lab}) = \ell$ be fresh. Define:

$$\begin{aligned}
\mathsf{Guard}(T_\tau)(\langle M, S\rangle, \chi, \iota) &\equiv v_r \notin \mathsf{Ids}(M, S) \ \wedge\ \ell \notin \mathsf{Labs}(M, S),\\
\mathsf{Out}(T_\tau)(\langle M, S\rangle, \chi, \iota) &= [\, p_{\mathsf{frz},\tau} \mapsto \{c_o\},\\
&\qquad p_{\mathsf{val},\mathsf{Ref}^{\mathsf{shr}}_{\ell_\bullet}(\tau)} \mapsto \{\langle v_r, \theta, \lambda[\ell_\bullet \mapsto \ell]\rangle\}\,],\\
\mathsf{Act}(T_\tau) &= \mathsf{push}(\mathsf{Freeze}(v_o) \cdot \mathsf{Shr}(v_o, v_r, \ell)).
\end{aligned}$$

(5) Additional shared borrow while already frozen. Add $\mathsf{BorrowShrFrz}_\tau$ as T_τ: $W(p_{\mathsf{frz},\tau}, T_\tau)$=$W(T_\tau, p_{\mathsf{frz},\tau})$=$W(T_\tau, p_{\mathsf{val},\mathsf{Ref}^{\mathsf{shr}}_{\ell_\bullet}(\tau)})$=1. Let $c_o = \langle v_o, \theta, \lambda\rangle = \mathsf{one}(\chi(p_{\mathsf{frz},\tau}))$ and let $v(\mathsf{ref}) = v_r$, $\mu(\mathsf{lab}) = \ell$ be fresh. Define:

$$\begin{aligned}
\mathsf{Guard}(T_\tau)(\langle M, S\rangle, \chi, \iota) &\equiv v_r \notin \mathsf{Ids}(M, S) \ \wedge\ \ell \notin \mathsf{Labs}(M, S),\\
\mathsf{Out}(T_\tau)(\langle M, S\rangle, \chi, \iota) &= [\, p_{\mathsf{frz},\tau} \mapsto \{c_o\},\\
&\qquad p_{\mathsf{val},\mathsf{Ref}^{\mathsf{shr}}_{\ell_\bullet}(\tau)} \mapsto \{\langle v_r, \theta, \lambda[\ell_\bullet \mapsto \ell]\rangle\}\,],\\
\mathsf{Act}(T_\tau) &= \mathsf{push}(\mathsf{Shr}(v_o, v_r, \ell)).
\end{aligned}$$

(6) Mutable borrow from an owned value. Add $\mathsf{BorrowMut}_\tau$ as T_τ: $W(p_{\mathsf{own},\tau}, T_\tau)$=$W(T_\tau, p_{\mathsf{blk},\tau})$=$W(T_\tau, p_{\mathsf{val},\mathsf{Ref}^{\mathsf{mut}}_{\ell_\bullet}(\tau)})$=1. Let $c_o = \langle v_o, \theta, \lambda\rangle = \mathsf{one}(\chi(p_{\mathsf{own},\tau}))$ and let $v(\mathsf{ref}) = v_r$, $\mu(\mathsf{lab}) = \ell$ be fresh. Define:

$$\begin{aligned}
\mathsf{Guard}(T_\tau)(\langle M, S\rangle, \chi, \iota) &\equiv v_r \notin \mathsf{Ids}(M, S) \ \wedge\ \ell \notin \mathsf{Labs}(M, S),\\
\mathsf{Out}(T_\tau)(\langle M, S\rangle, \chi, \iota) &= [\, p_{\mathsf{blk},\tau} \mapsto \{c_o\},\\
&\qquad p_{\mathsf{val},\mathsf{Ref}^{\mathsf{mut}}_{\ell_\bullet}(\tau)} \mapsto \{\langle v_r, \theta, \lambda[\ell_\bullet \mapsto \ell]\rangle\}\,],\\
\mathsf{Act}(T_\tau) &= \mathsf{push}(\mathsf{Mut}(v_o, v_r, \ell)).
\end{aligned}$$

3.6 Lifetimes via Stack Binding

(7) Ending a mutable borrow (restore owner). Add EndMut_τ as T_τ: $W(p_{\mathsf{blk},\tau}, T_\tau)$=$W(p_{\mathsf{val},\mathsf{Ref}^{\mathsf{mut}}_{\ell_\bullet}(\tau)}, T_\tau)$=$W(T_\tau, p_{\mathsf{own},\tau})$=1.

Let $c_o = \langle v_o, \theta_o, \lambda_o\rangle = \mathsf{one}(\chi(p_{\mathsf{blk},\tau}))$ and $c_r = \langle v_r, \theta_r, \lambda_r\rangle = \mathsf{one}(\chi(p_{\mathsf{val},\mathsf{Ref}^{\mathsf{mut}}_{\ell_\bullet}(\tau)}))$. Define:

$$\begin{aligned}
\mathsf{Guard}(T_\tau)(\langle M, S\rangle, \chi, \iota) &\equiv S = \mathsf{Mut}(v_o, v_r, \ell) \cdot S'\\
&\qquad \wedge\ \theta_o = \theta_r \ \wedge\ \lambda_o = \lambda_r,\\
\mathsf{Out}(T_\tau)(\langle M, S\rangle, \chi, \iota) &= [\, p_{\mathsf{own},\tau} \mapsto \{\langle v_o, \theta_o, \lambda_o\rangle\}\,],\\
\mathsf{Act}(T_\tau) &= \mathsf{pop}(\mathsf{Mut}(v_o, v_r, \ell)).
\end{aligned}$$

(8) Ending a shared borrow (non-last). Add $\mathsf{EndShrKeep}_\tau$ as T_τ: $W(p_{\mathsf{frz},\tau}, T_\tau)$=$W(p_{\mathsf{val},\mathsf{Ref}^{\mathsf{shr}}_{\ell_\bullet}(\tau)}, T_\tau)$=$W(T_\tau, p_{\mathsf{frz},\tau})$=1.

Let $c_o = \langle v_o, \theta, \lambda\rangle = \mathsf{one}(\chi(p_{\mathsf{frz},\tau}))$ and $c_r = \langle v_r, \theta', \lambda'\rangle = \mathsf{one}(\chi(p_{\mathsf{val},\mathsf{Ref}^{\mathsf{shr}}_{\ell_\bullet}(\tau)}))$. Define:

$$\begin{aligned}
\mathsf{Guard}(T_\tau)(\langle M, S\rangle, \chi, \iota) &\equiv S = \mathsf{Shr}(v_o, v_r, \ell) \cdot S'\\
&\qquad \wedge\ \theta = \theta' \ \wedge\ \lambda = \lambda'\\
&\qquad \wedge\ \big(S' = \epsilon \ \vee\ \mathsf{top}(S') \neq \mathsf{Freeze}(v_o)\big),\\
\mathsf{Out}(T_\tau)(\langle M, S\rangle, \chi, \iota) &= [\, p_{\mathsf{frz},\tau} \mapsto \{\langle v_o, \theta, \lambda\rangle\}\,],\\
\mathsf{Act}(T_\tau) &= \mathsf{pop}(\mathsf{Shr}(v_o, v_r, \ell)).
\end{aligned}$$

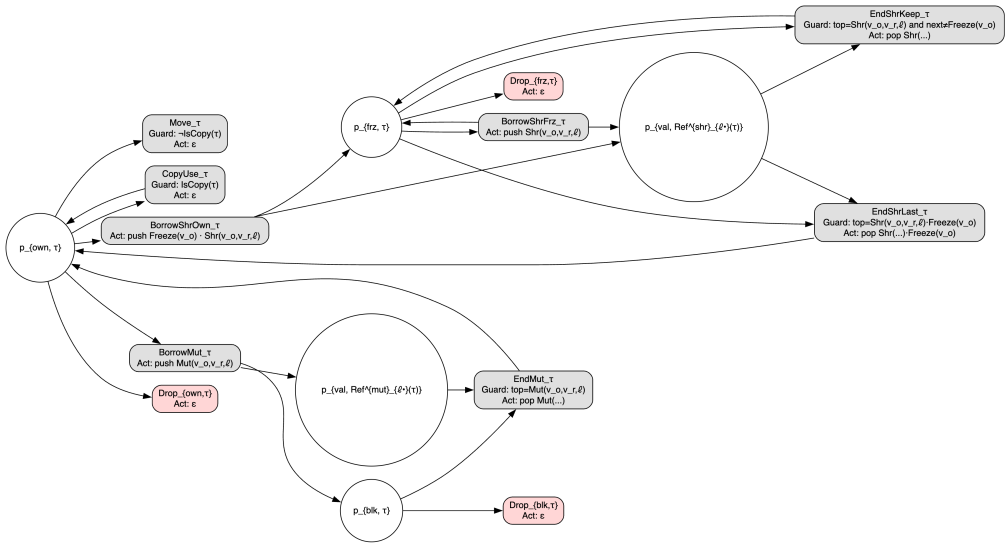

Figure 1: PCPN mapping: ownership + lifetimes (parametric in τ)

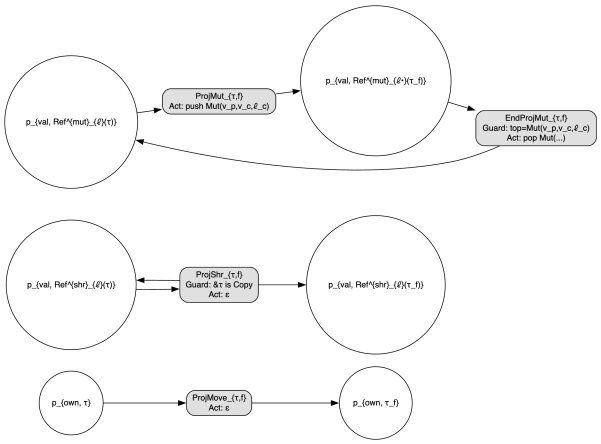

Figure 2: PCPN mapping: field projection & (re)borrow (struct τ has field $f : \tau_f$)

(9) Ending a shared borrow (last, restore owner). Add $\mathsf{EndShrLast}_\tau$ as T_τ: $W(p_{\mathsf{frz},\tau}, T_\tau)$=$W(p_{\mathsf{val},\mathsf{Ref}^{\mathsf{shr}}_{\ell_\bullet}(\tau)}, T_\tau)$=$W(T_\tau, p_{\mathsf{own},\tau})$=1.

Let $c_o = \langle v_o, \theta, \lambda\rangle = \mathsf{one}(\chi(p_{\mathsf{frz},\tau}))$ and $c_r = \langle v_r, \theta', \lambda'\rangle = \mathsf{one}(\chi(p_{\mathsf{val},\mathsf{Ref}^{\mathsf{shr}}_{\ell_\bullet}(\tau)}))$. Define:

$$\begin{aligned}
\mathsf{Guard}(T_\tau)(\langle M, S\rangle, \chi, \iota) &\equiv S = \mathsf{Shr}(v_o, v_r, \ell) \cdot \mathsf{Freeze}(v_o) \cdot S' \\
&\quad \wedge\ \theta = \theta' \ \wedge\ \lambda = \lambda', \\
\mathsf{Out}(T_\tau)(\langle M, S\rangle, \chi, \iota) &= [\, p_{\mathsf{own},\tau} \mapsto \{\langle v_o, \theta, \lambda\rangle\} \,], \\
\mathsf{Act}(T_\tau) &= \mathsf{pop}(\mathsf{Shr}(v_o, v_r, \ell) \cdot \mathsf{Freeze}(v_o)).
\end{aligned}$$

3.7 Composite Types and Field (Re)Borrowing

(10) Field projection by move (conservative). Let τ be a struct type with field $f : \tau_f$. Add $\mathsf{ProjMove}_{\tau,f}$ as $T_{\tau,f}$:

$$W(p_{\mathsf{own},\tau}, T_{\tau,f}) = W(T_{\tau,f}, p_{\mathsf{own},\tau_f}) = 1.$$

Let $c = \langle v, \theta, \lambda\rangle = \mathsf{one}(\chi(p_{\mathsf{own},\tau}))$ and let $v(\mathsf{fld}) = v_f$ be fresh. Define:

$$\begin{aligned}
\mathsf{Guard}(T_{\tau,f})(\langle M, S\rangle, \chi, \iota) &\equiv v_f \notin \mathsf{Ids}(M, S), \\
\mathsf{Out}(T_{\tau,f})(\langle M, S\rangle, \chi, \iota) &= [\, p_{\mathsf{own},\tau_f} \mapsto \{\langle v_f, \theta, \lambda\rangle\} \,], \\
\mathsf{Act}(T_{\tau,f}) &= \epsilon.
\end{aligned}$$

(11) Field reborrow from a shared reference (Copy, keep parent). Add $\mathsf{ProjShr}_{\tau,f}$ as T_τ:

$$W(p_{\mathsf{val},\mathsf{Ref}^{\mathsf{shr}}_{\ell}(\tau)}, T_{\tau,f}) = W(T_{\tau,f}, p_{\mathsf{val},\mathsf{Ref}^{\mathsf{shr}}_{\ell}(\tau)}) = W(T_{\tau,f}, p_{\mathsf{val},\mathsf{Ref}^{\mathsf{shr}}_{\ell}(\tau_f)}) = 1.$$

Let $c_p = \langle v_p, \theta, \lambda\rangle = \mathsf{one}(\chi(p_{\mathsf{val},\mathsf{Ref}^{\mathsf{shr}}_{\ell}(\tau)}))$ and let $v(\mathsf{child}) = v_c$ be fresh. Define:

$$\begin{aligned}
\mathsf{Guard}(T_{\tau,f})(\langle M, S\rangle, \chi, \iota) &\equiv v_c \notin \mathsf{Ids}(M, S), \\
\mathsf{Out}(T_{\tau,f})(\langle M, S\rangle, \chi, \iota) &= [\, p_{\mathsf{val},\mathsf{Ref}^{\mathsf{shr}}_{\ell}(\tau)} \mapsto \{c_p\}, \\
&\qquad p_{\mathsf{val},\mathsf{Ref}^{\mathsf{shr}}_{\ell}(\tau_f)} \mapsto \{\langle v_c, \theta, \lambda\rangle\} \,], \\
\mathsf{Act}(T_{\tau,f}) &= \epsilon.
\end{aligned}$$

(12) Field reborrow from a mutable reference (reborrowing, block parent ref). Add $\mathsf{ProjMut}_{\tau,f}$ as T_τ:

$$W(p_{\mathsf{val},\mathsf{Ref}^{\mathsf{mut}}_{\ell}(\tau)}, T_{\tau,f}) = W(T_{\tau,f}, p_{\mathsf{val},\mathsf{Ref}^{\mathsf{mut}}_{\ell_\bullet}(\tau_f)}) = 1.$$

Let $c_p = \langle v_p, \theta, \lambda \rangle = \mathsf{one}(\chi(p_{\mathsf{val},\mathsf{Ref}_\ell^{\mathsf{mut}}(\tau)}))$ and let $\nu(\mathsf{child}) = v_c$, $\mu(\mathsf{lab}) = \ell_c$ be fresh. Define:

$$\begin{aligned}
\mathsf{Guard}(T_{\tau,f})(\langle M, S\rangle, \chi, \iota) &\equiv v_c \notin \mathsf{Ids}(M,S) \ \wedge\ \ell_c \notin \mathsf{Labs}(M,S),\\
\mathsf{Out}(T_{\tau,f})(\langle M, S\rangle, \chi, \iota) &= [\, p_{\mathsf{val},\mathsf{Ref}_{\ell_\bullet}^{\mathsf{mut}}(\tau_f)} \mapsto \{\langle v_c, \theta, \lambda[\ell_\bullet \mapsto \ell_c]\rangle\}\,],\\
\mathsf{Act}(T_{\tau,f}) &= \mathsf{push}(\mathsf{Mut}(v_p, v_c, \ell_c)).
\end{aligned}$$

(13) Ending a mutable reborrow (restore parent ref). Add $\mathsf{EndProjMut}_{\tau,f}$ as T_τ:

$$W(p_{\mathsf{val},\mathsf{Ref}_{\ell_\bullet}^{\mathsf{mut}}(\tau_f)}, \mathsf{EndProjMut}_{\tau,f}) = W(\mathsf{EndProjMut}_{\tau,f}, p_{\mathsf{val},\mathsf{Ref}_\ell^{\mathsf{mut}}(\tau)}) = 1.$$

Let $c_c = \langle v_c, \theta, \lambda \rangle = \mathsf{one}(\chi(p_{\mathsf{val},\mathsf{Ref}_{\ell_\bullet}^{\mathsf{mut}}(\tau_f)}))$. Define:

$$\begin{aligned}
\mathsf{Guard}(T_{\tau,f})(\langle M, S\rangle, \chi, \iota) &\equiv S = \mathsf{Mut}(v_p, v_c, \ell_c) \cdot S',\\
\mathsf{Out}(T_{\tau,f})(\langle M, S\rangle, \chi, \iota) &= [\, p_{\mathsf{val},\mathsf{Ref}_\ell^{\mathsf{mut}}(\tau)} \mapsto \{\langle v_p, \theta, \lambda\rangle\}\,],\\
\mathsf{Act}(T_{\tau,f}) &= \mathsf{pop}(\mathsf{Mut}(v_p, v_c, \ell_c)).
\end{aligned}$$

3.8 Traits and Associated Types

(14) Trait witness production. If $\Sigma(\mathcal{C}) \vDash \tau : \mathsf{Tr}$, add $\mathsf{Impl}_{\tau,\mathsf{Tr}}$:

$$W(p_{\mathsf{own},\tau}, \mathsf{Impl}_{\tau,\mathsf{Tr}}) = W(\mathsf{Impl}_{\tau,\mathsf{Tr}}, p_{\mathsf{own},\tau}) = 1$$

$W(\mathsf{Impl}_{\tau,\mathsf{Tr}}, p_{\mathsf{val},\mathsf{Impl}(\tau,\mathsf{Tr})}) = 1.$
Let $c = \langle v, \theta, \lambda \rangle = \mathsf{one}(\chi(p_{\mathsf{own},\tau}))$. Define:

$$\begin{aligned}
\mathsf{Guard}(\mathsf{Impl}_{\tau,\mathsf{Tr}})(\langle M, S\rangle, \chi, \iota) &\equiv \Sigma(\mathcal{C}) \vDash \tau : \mathsf{Tr},\\
\mathsf{Out}(\mathsf{Impl}_{\tau,\mathsf{Tr}})(\langle M, S\rangle, \chi, \iota) &= [\, p_{\mathsf{own},\tau} \mapsto \{c\},\\
&\qquad p_{\mathsf{val},\mathsf{Impl}(\tau,\mathsf{Tr})} \mapsto \{\langle v, \theta, \lambda\rangle\}\,],\\
\mathsf{Act}(\mathsf{Impl}_{\tau,\mathsf{Tr}}) &= \epsilon.
\end{aligned}$$

(15) Associated type resolution as retyping. If $\Sigma(\mathcal{C}) \vDash \mathsf{Assoc}(\tau, \mathsf{Tr}, A) = \tau'$, add $\mathsf{AssocCast}_{\tau,\mathsf{Tr},A}$:

$$W(p_{\mathsf{val},\mathsf{Assoc}(\tau,\mathsf{Tr},A)}, \mathsf{AssocCast}_{\tau,\mathsf{Tr},A}) = W(\mathsf{AssocCast}_{\tau,\mathsf{Tr},A}, p_{\mathsf{val},\tau'}) = 1.$$

Let $c = \mathsf{one}(\chi(p_{\mathsf{val},\mathsf{Assoc}(\tau,\mathsf{Tr},A)}))$. Define:

$$\begin{aligned}
\mathsf{Guard}(\mathsf{AssocCast}_{\tau,\mathsf{Tr},A})(\langle M, S\rangle, \chi, \iota) &\equiv \Sigma(\mathcal{C}) \vDash \mathsf{Assoc}(\tau, \mathsf{Tr}, A) = \tau',\\
\mathsf{Out}(\mathsf{AssocCast}_{\tau,\mathsf{Tr},A})(\langle M, S\rangle, \chi, \iota) &= [\, p_{\mathsf{val},\tau'} \mapsto \{c\}\,],\\
\mathsf{Act}(\mathsf{AssocCast}_{\tau,\mathsf{Tr},A}) &= \epsilon.
\end{aligned}$$

3.9 Soundness Intuition

By construction, any transition sequence in $\mathcal{N}_{\mathcal{C}}$ corresponds to a harness in which: (i) moved owned values are not reused (they are consumed from $p_{\mathsf{own},\tau}$), (ii) mutable aliasing is prevented (owners are moved to $p_{\mathsf{blk},\tau}$ under Mut frames), (iii) mutation under shared aliasing is prevented (owners reside in $p_{\mathsf{frz},\tau}$ under Freeze/Shr frames), and (iv) lifetimes are properly nested and ended through stack-pop transitions. Generic instantiation is guarded by the satisfiability relation $\Vdash$ and associated-type resolution. Therefore, generated harnesses are expected to be accepted by the Rust compiler under this conservative, structured borrowing discipline.

4 Formal Analysis for Sequence Generation and Coverage

4.1 Executions, Traces, and Snippet Well-Formedness

A *trace* is a finite sequence of firing certificates $\sigma = \kappa_1 \cdots \kappa_n$ such that $\mathsf{Cfg}_0[\sigma\rangle\mathsf{Cfg}_n$ where $\mathsf{Cfg}_0 = \langle M_0, \epsilon\rangle$ and the step relation $\mathsf{Cfg}[\kappa\rangle\mathsf{Cfg}'$ follows Definition 2.

We write $\mathsf{Emit}(\sigma)$ for the Rust snippet synthesized from σ using the API transitions emit calls; structural transitions emit borrows, drops, projections, and reborrow scopes.

Closed configurations. Define:

$$\mathsf{Closed}(\langle M, S\rangle) \ \triangleq\ (S = \epsilon).$$

Optionally, one may strengthen the notion by additionally requiring that no owner remains frozen/blocked: $\forall \tau : M(p_{\mathsf{frz},\tau}) = \emptyset \ \wedge\ M(p_{\mathsf{blk},\tau}) = \emptyset$.

Well-formed snippet traces. A trace σ is *snippet-well-formed* if it can be extended to a closed configuration:

$$\exists \sigma' : \mathsf{Cfg}_0[\sigma \cdot \sigma'\rangle\mathsf{Cfg} \ \wedge\ \mathsf{Closed}(\mathsf{Cfg}).$$

In practice, we build σ' by deterministically closing the push-down stack (Section 4.7).

4.2 Reachability Graph vs. Coverability Tree

In Petri-net analysis, *reachability graphs* provide an exact state-space for bounded systems, whereas *coverability trees* (e.g., Karp–Miller) over-approximate unbounded systems and may introduce ω-markings. Since our goal is to synthesize *concrete* firing sequences that translate to *concrete* Rust snippets, we base our analysis on a canonical reachability graph (CRG). For trace extraction we additionally maintain a *witness tree* as a by-product of reachability saturation.

4.3 Guard Solving and Certificate Enumeration

For each transition $t \in T$, we write the guard as a conjunction:

$$\begin{aligned}
\mathsf{Guard}(t)(\mathsf{Cfg}, \chi, \iota) \ \equiv\ & G_t^{\mathsf{uni}}(\chi, \iota) \ \wedge\ G_t^{\mathsf{life}}(\chi, \mathsf{Cfg}.S, \iota)\\
& \wedge\, G_t^{\mathsf{trait}}(\chi, \iota) \ \wedge\ G_t^{\mathsf{cap}}(\chi, \mathsf{Cfg}.S, \iota).
\end{aligned}$$

(1) Type unification G_t^{uni}. It enforces consistent instantiation of generic type variables across all consumed token colors and signature-imposed equalities. Concretely, ι contains a substitution θ and G_t^{uni} requires that θ agrees with token-carried substitutions and that all type equalities in the signature/where-clause hold after applying θ.

(2) Lifetime/stack constraint G_t^{life}. It enforces lifetime substitution λ consistency and stack compatibility: (i) if a signature uses the same lifetime variable across multiple parameters, their chosen tokens must agree on λ, and (ii) if t pops frames, the required pattern must match the stack top.

Algorithm 1 SolveGuard(t, Cfg, χ)

Require: transition t, configuration Cfg $= \langle M, S\rangle$, choice χ
Ensure: finite set of admissible instantiations Inst(t, Cfg, χ)

```
Θ ← UnifyTypes(t, χ)
Λ ← UnifyLifetimes(t, χ, S)
Inst ← ∅
for all (θ, λ) ∈ Θ × Λ do
    if CheckTraitAssoc(t, θ) and CheckCap(t, Cfg, χ, θ, λ) then
        Inst ← Inst ∪ {(θ, λ, ...)}
    end if
end for
return Inst =0
```

(3) Trait/associated-type constraint G_t^{trait}. It checks all bounds in the where-clause under θ: for each requirement ρ_i, require $\theta(\tau) \Vdash \rho_i$ and that associated-type equalities resolve consistently.

(4) Capability constraint G_t^{cap}. It enforces Rust-compatibility of consumed/produced places including ownership/borrow discipline such as borrows move owners to frz/blk, mutable effects cannot directly consume from blocked owners, etc.

4.4 Boundedness and Finiteness of the Analysis Domain

Budgeted boundedness (constructive). Let $B : P \to \mathbb{N}$ be a *place budget*. We enforce B by requiring that any transition that would increase tokens in p satisfies $|M(p)| < B(p)$ in its guard (e.g., constructors, copy-duplication, API calls that return fresh owned values). We similarly enforce a *stack-depth budget* $D \in \mathbb{N}$ by requiring $|S| < D$ before any push action.

Finite instantiation domain. For each generic type variable α, let Cand(α) $\subseteq$ GTy be a finite candidate set computed from the crate (e.g., monomorphic types appearing in $\Sigma(\mathcal{C})$). Then UnifyTypes returns a finite set of θ.

Canonicalization (quotient by α-renaming). Let Canon($\langle M, S\rangle$) rename all value identifiers and region labels to a canonical sequence based on first occurrence (and canonically order multisets), yielding a representative of the α-equivalence class.

[Finiteness assumptions] We assume:

(B1) token counts are bounded by a budget B (or equivalently, the net is k-bounded);
(B2) stack depth is bounded by D;
(B3) each generic variable ranges over a finite candidate set Cand(α), hence SolveGuard is finite;
(B4) configurations are quotiented by canonicalization Canon.

Theorem 1 (Finite canonical state space). *Under Assumption 4.4, there are only finitely many canonical configurations* $\widehat{\text{Cfg}} = \text{Canon}(\text{Cfg})$ *reachable from* Canon($\langle M_0, \epsilon\rangle$).

Proof sketch. (B1) bounds token multiplicities per place; since P is finite for a fixed crate, the multiset carrier is bounded. (B2) bounds stack words over a finite alphabet. (B3) bounds instantiation choices per step. (B4) quotients away unboundedly many fresh names. Thus the canonical reachable set is finite. □

Algorithm 2 SaturateWithWitness($\mathcal{N}_{\mathcal{C}}$)

```
V̂ ← ∅, Ê ← ∅
parent(·) ← ⊥
Ĉfg_0 ← Canon(⟨M_0, ε⟩)
W ← {Ĉfg_0}
while W ≠ ∅ do
    remove some Ĉfg = ⟨M̂, Ŝ⟩ from W
    if Ĉfg ∈ V̂ then
        continue
    end if
    V̂ ← V̂ ∪ {Ĉfg}
    for all t ∈ T do
        for all choices χ for •t at M̂ do
            for all ι ∈ SolveGuard(t, Ĉfg, χ) do
                κ ← ⟨t, χ, ι⟩
                Ĉfg′ ← Canon(Fire(Ĉfg, κ))
                Ê ← Ê ∪ {(Ĉfg, κ, Ĉfg′)}
                if Ĉfg′ ∉ V̂ and parent(Ĉfg′) = ⊥ then
                    parent(Ĉfg′) ← (Ĉfg, κ)
                end if
                W ← W ∪ {Ĉfg′}
            end for
        end for
    end for
end while
return (V̂, Ê, parent) =0
```

4.5 Canonical Reachability Graph and Witness Tree

Definition 3 (Annotated Canonical Reachability Graph). *Let* $\widehat{\text{Cfg}}_0 = \text{Canon}(\langle M_0, \epsilon\rangle)$. *The canonical reachability graph is* $\widehat{G} = (\widehat{V}, \widehat{E})$ *where* $\widehat{V}$ *is the set of canonical configurations reachable from* $\widehat{\text{Cfg}}_0$, *and*

$$(\widehat{\text{Cfg}}, \kappa, \widehat{\text{Cfg}}') \in \widehat{E} \quad \textit{iff} \quad \exists \text{Cfg},\ \text{Cfg}' : \text{Canon}(\text{Cfg}) = \widehat{\text{Cfg}},\ \text{Canon}(\text{Cfg}') = \widehat{\text{Cfg}}',\ \text{Cfg}[\kappa\rangle\text{Cfg}'.$$

Definition 4 (Witness tree). *A witness tree* $\mathcal{T}$ *is a rooted spanning tree over* $\widehat{V}$ *that stores, for each discovered node* $\widehat{\text{Cfg}} \neq \widehat{\text{Cfg}}_0$, *one parent edge* $(\widehat{\text{Par}}, \kappa, \widehat{\text{Cfg}}) \in \widehat{E}$. *Thus each node has a unique root-to-node witness trace.*

Theorem 2 (Termination and completeness of saturation). *Under Assumption 4.4, Algorithm 2 terminates and returns the complete canonical reachability graph* $\widehat{G}$ *and a witness tree spanning* $\widehat{V}$.

Algorithm 3 AnalyzeDeadQuasiLive($\widehat{G}$)

Require: $\widehat{G} = (\widehat{V}, \widehat{E})$
Ensure: Dead, QuasiLive $\subseteq T$

```
Seen ← ∅
for all (Cfĝ, κ, Cfĝ′) ∈ Ê do
    Seen ← Seen ∪ {κ.tr}
end for
QuasiLive ← Seen
Dead ← T \ Seen
return (Dead, QuasiLive) =0
```

Proof sketch. By Theorem 1, there are finitely many canonical configurations. For each configuration, the number of (choices χ) is finite because token multiplicities are bounded, and the number of instantiations is finite by (B3). Thus saturation terminates. Completeness follows from enumerating all enabled certificates. □

Complexity note (optional). Let $\mathsf{Ch}(\widehat{M}, t)$ be the number of valid choices for t at marking $\widehat{M}$, and $\mathsf{Inst}(\widehat{\mathsf{Cfg}}, t, \chi)$ the number of admissible instantiations. The worst-case saturation cost is $O\Big(\sum_{\widehat{\mathsf{Cfg}} \in \widehat{V}} \sum_{t \in T} \sum_{\chi} |\mathsf{Inst}(\widehat{\mathsf{Cfg}}, t, \chi)|\Big)$.

4.6 Deadness and Liveness Analysis on $\widehat{G}$

Enabledness sets. For a transition t, define the set of nodes where t is enabled:

$$\mathsf{En}(t) = \{\, \widehat{\mathsf{Cfg}} \in \widehat{V} \mid \exists \kappa, \widehat{\mathsf{Cfg}}' \;:\; (\widehat{\mathsf{Cfg}}, \kappa, \widehat{\mathsf{Cfg}}') \in \widehat{E} \wedge \kappa.\mathsf{tr} = t \,\}.$$

Definition 5 (Dead, quasi-live, live). *Let $t \in T$.*

- *t is* dead *iff* $\mathsf{En}(t) = \emptyset$ *(no reachable firing instance).*
- *t is* quasi-live *iff* $\mathsf{En}(t) \neq \emptyset$ *(fireable at least once along some execution).*
- *t is* live *iff for every $\widehat{\mathsf{Cfg}} \in \widehat{V}$ there exists a path in $\widehat{G}$ from $\widehat{\mathsf{Cfg}}$ to some node in* $\mathsf{En}(t)$.

Deadlocks and termination sinks. A canonical configuration $\widehat{\mathsf{Cfg}}$ is a *deadlock* iff it has no outgoing edges in $\widehat{E}$. Such nodes can be detected by scanning out-degrees in $\widehat{G}$.

4.7 Closing Borrows and Making Snippets Self-Contained

Lemma 1 (Closing terminates). *If* CloseStack *does not get stuck, it terminates in at most $|S|$ iterations and returns a trace that empties the stack.*

Proof sketch. Each step pops at least one stack symbol; hence the stack length strictly decreases. □

Pragmatic note. In our construction, each push action is paired with a corresponding ending transition that restores owners/references, so reachable configurations admit stack closing under the intended usage discipline (structured borrows).

Algorithm 4 AnalyzeLiveness($\widehat{G}, t$)

Require: $\widehat{G} = (\widehat{V}, \widehat{E})$, transition t
Ensure: boolean IsLive(t)

```
Targets ← En(t)
if Targets = ∅ then
    return false
end if
build reverse adjacency Ê^{-1}
R ← Targets {nodes that can reach an enabling node}
Q ← Targets
while Q ≠ ∅ do
    pop Cfĝ from Q
    for all (Cfĝ_pre, _, Cfĝ) ∈ Ê^{-1} do
        if Cfĝ_pre ∉ R then
            R ← R ∪ {Cfĝ_pre}
            Q ← Q ∪ {Cfĝ_pre}
        end if
    end for
end while
return (R = V̂) =0
```

Algorithm 5 CloseStack(Cfg)

Require: reachable configuration $\mathsf{Cfg} = \langle M, S \rangle$
Ensure: a closing trace σ_{close} such that $\mathsf{Cfg}[\sigma_{\text{close}}\rangle\langle M', \epsilon\rangle$

```
σ_close ← ε
while S ≠ ε do
    let γ = top(S)
    choose the unique applicable ending transition t_end for γ (by pattern)
    choose χ matching the required input tokens for t_end
    pick some ι ∈ SolveGuard(t_end, ⟨M, S⟩, χ)
    fire κ = ⟨t_end, χ, ι⟩ to update (M, S)
    σ_close ← σ_close · κ
end while
return σ_close =0
```

4.8 Coverage Metrics over Snippet Sets

Let $T_{\text{API}} \subseteq T$ be transitions corresponding to crate functions/methods. Define the *reachable* API transition set:

$$T_{\text{API}}^{\text{reach}} = \{\, t \in T_{\text{API}} \mid \mathsf{En}(t) \neq \emptyset \,\}.$$

For type forms, let $P_{\text{form}} \subseteq P$ be places representing type forms of interest (owned/ref/assoc/field/witness, etc.), and define:

$$P_{\text{form}}^{\text{reach}} = \{\, p \in P_{\text{form}} \mid \exists \widehat{\mathsf{Cfg}} = \langle \widehat{M}, \hat{S} \rangle \in \widehat{V} \;:\; \widehat{M}(p) \neq \emptyset \,\}.$$

For borrow shapes, define:

$$\mathcal{B}^{\text{reach}} = \{\, \mathsf{Shape}(\hat{S}) \mid \langle \widehat{M}, \hat{S} \rangle \in \widehat{V} \,\}.$$

Coverage of a finite trace set. Let Σ be a finite set of traces (equivalently, a set of generated snippets $\{\mathsf{Emit}(\sigma) \mid \sigma \in \Sigma\}$).

Define:

$$\mathsf{API}(\Sigma) = \bigcup_{\sigma\in\Sigma}\{\,\kappa.\mathrm{tr} \mid \kappa \in \sigma,\ \kappa.\mathrm{tr} \in T_{\mathrm{API}}\,\}.$$

Let σ induce intermediate configurations $\mathrm{Cfg}_i = \langle M_i, S_i\rangle$. Define:

$$\mathsf{Vis}_P(\Sigma) = \bigcup_{\sigma\in\Sigma}\bigcup_i\{\,p \in P_{\mathrm{form}} \mid M_i(p) \neq \emptyset\,\}, \quad \mathsf{Vis}_B(\Sigma) = \bigcup_{\sigma\in\Sigma}\bigcup_i\{\mathsf{Shape}(S_i)\}.$$

Then:

$$\mathrm{Cov}_{\mathrm{API}}(\Sigma) = \frac{|\mathsf{API}(\Sigma)|}{|T_{\mathrm{API}}^{\mathrm{reach}}|},$$

$$\mathrm{Cov}_{\mathrm{Type}}(\Sigma) = \frac{|\mathsf{Vis}_P(\Sigma) \cap P_{\mathrm{form}}^{\mathrm{reach}}|}{|P_{\mathrm{form}}^{\mathrm{reach}}|},$$

$$\mathrm{Cov}_{\mathrm{Borrow}}(\Sigma) = \frac{|\mathsf{Vis}_B(\Sigma)|}{|\mathcal{B}^{\mathrm{reach}}|}.$$

We combine them as:

$$\mathrm{GCov}(\Sigma) = w_1{\cdot}\mathrm{Cov}_{\mathrm{API}}(\Sigma)+w_2{\cdot}\mathrm{Cov}_{\mathrm{Type}}(\Sigma)+w_3{\cdot}\mathrm{Cov}_{\mathrm{Borrow}}(\Sigma), \quad w_i \geq 0.$$

4.9 Coverage-Guaranteed Snippet Generation from $\widehat{G}$

Witness extraction. Using the witness-tree parent map, each reachable node $\widehat{\mathrm{Cfg}}$ has a unique witness trace $\mathsf{WitNode}(\widehat{\mathrm{Cfg}})$ obtained by backtracking parent pointers to the root. For each edge $(\widehat{\mathrm{Cfg}}, \kappa, \widehat{\mathrm{Cfg}}') \in \widehat{E}$, define:

$$\mathsf{WitEdge}(\widehat{\mathrm{Cfg}}, \kappa, \widehat{\mathrm{Cfg}}') = \mathsf{WitNode}(\widehat{\mathrm{Cfg}}) \cdot \kappa.$$

Theorem 3 (Full model-level coverage within bounds). *Let $(\widehat{V}, \widehat{E}, \mathrm{parent})$ be the output of Algorithm 2. Define:*

$$\Sigma_{\mathrm{API}} = \{\,\mathsf{WitEdge}(e) \mid e \in \widehat{E},\ e.\kappa.\mathrm{tr} \in T_{\mathrm{API}}\,\}.$$

Then $\mathrm{Cov}_{\mathrm{API}}(\Sigma_{\mathrm{API}}) = 1$. Moreover, by adding witnesses for first occurrences of each $p \in P_{\mathrm{form}}^{\mathrm{reach}}$ and each $b \in \mathcal{B}^{\mathrm{reach}}$, we obtain a finite set $\Sigma^\star$ such that

$$\mathrm{Cov}_{\mathrm{API}}(\Sigma^\star) = \mathrm{Cov}_{\mathrm{Type}}(\Sigma^\star) = \mathrm{Cov}_{\mathrm{Borrow}}(\Sigma^\star) = 1.$$

Proof sketch. For any $t \in T_{\mathrm{API}}^{\mathrm{reach}}$, there exists an edge in $\widehat{E}$ whose certificate transition is t. Its witness edge trace contains t. The place/shape statements follow by selecting a node/edge where the objective appears. □

Making witnesses self-contained. For each witness trace σ, we post-process it as $\sigma^{\downarrow} = \sigma \cdot \mathsf{CloseStack}(\mathrm{Cfg}_\sigma)$ where Cfg_σ is the reached configuration after firing σ. Then $\mathsf{Emit}(\sigma^{\downarrow})$ yields a self-contained snippet.

4.10 Budgeted High-Coverage Selection

Let $\mathcal{S}$ be a pool of candidate traces (e.g., witnesses for edges or for objectives). Each trace σ covers objectives:

$$\mathsf{Obj}(\sigma) = \big(\mathsf{API}(\{\sigma\})\big) \cup \big(\mathsf{Vis}_P(\{\sigma\}) \cap P_{\mathrm{form}}^{\mathrm{reach}}\big) \cup \mathsf{Vis}_B(\{\sigma\}).$$

Given a budget m, select $\Sigma \subseteq \mathcal{S}$ with $|\Sigma| \leq m$ maximizing $\big|\bigcup_{\sigma\in\Sigma}\mathsf{Obj}(\sigma)\big|$.

Algorithm 6 GreedyCover($\mathcal{S}, m$)

```
Σ ← ∅, U ← all objectives initially uncovered
for i = 1 to m do
    choose σ* ∈ S \ Σ maximizing |Obj(σ*) ∩ U|
    Σ ← Σ ∪ {σ*}
    U ← U \ Obj(σ*)
end for
return Σ =0
```

Optional weighted variant. If each objective o has weight $w(o)$ and each trace has cost $\mathrm{cost}(\sigma)$ (e.g., length), we can use a standard greedy ratio rule maximizing $\frac{w(\mathsf{Obj}(\sigma)\cap U)}{\mathrm{cost}(\sigma)}$.

5 Evaluation

5.1 Research Questions

We structure the evaluation around the following research questions:

- **RQ1 (Compilability & Executability).** Does our PCPN-guided generator produce compilable and executable harnesses at a higher rate than baselines that do not model ownership/borrowing/lifetimes/generics precisely?
- **RQ2 (Target Code Coverage).** Under the same fuzzing budget, do PCPN-generated harnesses achieve higher *target-library* coverage (line/branch/function coverage of the crate-under-test) than competing approaches?
- **RQ3 (Bug-Finding Effectiveness).** Does our approach find more unique failures (e.g., panics, assertion violations, sanitizer crashes) and/or find them faster (time-to-first-crash) than baselines?
- **RQ4 (Feature Support and Stress Cases).** How robust is the approach on crates that heavily use (i) borrow-carrying types and explicit lifetimes in signatures, (ii) generic APIs with trait bounds and associated types, and (iii) composite types requiring field projection/reborrowing?
- **RQ5 (Ablation: Which Components Matter?).** What is the contribution of each major component (pushdown lifetime stack, generic partial-order instantiation, field (re)borrowing rules, POR/heuristics if any) to compilability and coverage?
- **RQ6 (Model Coverage → Code Coverage).** To what extent does increasing *net-level coverage* (e.g., transition coverage over API transitions) correlate with increased *target-code* coverage, and can it serve as a reliable surrogate objective?

5.2 Benchmarks and Subjects

Real-world crates. We evaluate on a curated set of Rust crates drawn from crates.io. We select crates that (i) expose non-trivial public APIs (multiple modules/types), (ii) include stateful protocols (resources/handles/transactions/builders), and (iii) collectively cover diverse domains (parsers, codecs, data structures, storage, networking, etc.). For each crate, we extract

callable items $\mathcal{F}$ (public functions/methods) and their signatures via the compiler front-end.

Feature-stratified subsets. We partition crates into groups based on signature-level features:

- **Lifetime-heavy**: explicit lifetimes and borrow-carrying return types in signatures;
- **Generic-heavy**: frequent type parameters with trait bounds / where-clauses / associated types;
- **Composite-heavy**: deep nested ADTs and frequent field projections (struct/tuple) in APIs.

5.3 Baselines

We compare against the following baselines:

B0: Byte-oriented fuzzing harness. A conventional setup where the fuzzer mutates a byte array and a harness decodes it into inputs for a fixed entry point.

B1: Signature-driven random sequences (no borrow/lifetime model). A type-directed generator that samples call sequences using signatures, but treats values as freely duplicable or ends borrows heuristically without a formal stack discipline.

B2: Petri net without pushdown (finite-state approximation). A Petri-net-based generator that uses places/transitions derived from signatures but replaces the pushdown lifetime mechanism with a finite-state approximation (e.g., lexical-only or bounded non-nested borrow tracking).

B3 (optional): Property-based testing. A property-based generator (e.g., arbitrary structured values) for selected APIs, without stateful call-sequence generation.

5.4 Metrics

Compilability and executability. Let $\mathcal{H}$ be the set of synthesized harnesses. We define:

$$\mathbf{CHR} = \frac{|\{h \in \mathcal{H} \mid \mathrm{rustc}(h) \text{ accepts}\}|}{|\mathcal{H}|}, \tag{1}$$

$$\mathbf{EHR} = \frac{|\{h \in \mathcal{H} \mid h \text{ executes and completes initialization}\}|}{|\mathcal{H}|}. \tag{2}$$

Target-library coverage. For each crate-under-test, we measure standard code coverage (e.g., line and branch coverage):

$$\mathbf{TLC} = \frac{\text{covered lines in target crate}}{\text{total instrumented lines in target crate}}, \tag{3}$$

$$\mathbf{TBC} = \frac{\text{covered branches in target crate}}{\text{total instrumented branches in target crate}}. \tag{4}$$

Model (net-level) coverage. Let $T_{\mathsf{api}} \subseteq T$ be transitions corresponding to extracted API calls. For a trace multiset Σ (generated during exploration/fuzzing), define:

$$\mathbf{NTC} = \frac{|\{t \in T_{\mathsf{api}} \mid \exists \sigma \in \Sigma : t \in \sigma\}|}{|T_{\mathsf{api}}|}. \tag{5}$$

Table 1: Benchmark summary (to be filled).

Crate	LOC	#API items $\lvert\mathcal{F}\rvert$	Lifetime-heavy?	Generic-heavy?

We record: (i) the number of unique failures (deduplicated by stack trace hash or sanitizer report signature), and (ii) time-to-first-crash (TTFC).

Cost and scalability. We measure: net construction time, exploration time per trace, harness compilation time, and fuzzing throughput (exec/s).

5.5 Experimental Protocol

For each crate and each approach (ours and baselines), we perform the following steps:

(1) **Model extraction (ours / B2).** Extract signatures $\mathcal{F}$ and build the Petri-net instance.
(2) **Harness generation.** Generate K harnesses (or K traces), each bounded by max trace length L and stack depth D.
(3) **Compilation filter.** Compile each harness; record CHR and compile-time.
(4) **Fuzzing run.** Run each compiled harness for a fixed time budget B_{time} (or fixed total executions), using the same fuzzer backend and identical resource limits.
(5) **Measurement.** Collect target coverage (TLC/TBC), net-level coverage (NTC), and failures.

Repetitions and statistics. Each configuration is repeated R times with independent random seeds. We report mean and standard deviation (or median and IQR) and use non-parametric tests when appropriate.

5.6 Ablation and Sensitivity Studies

Ablations. We evaluate the following variants:

- **NoStack**: remove the pushdown lifetime stack and replace with a finite approximation;
- **NoPO**: replace the partial-order-based generic instantiation with random instantiation from the same candidate pool;
- **NoFieldReborrow**: disable field projection/reborrow transitions;
- **NoAssoc**: disable associated-type rewriting transitions.

We compare each variant against the full system in CHR/EHR, TLC/TBC, and TTFC.

Sensitivity. We vary L (trace length bound), D (stack depth bound), and $|\mathsf{Cand}(\alpha)|$ (generic candidate pool size) to quantify trade-offs between exploration power, compilation success, and runtime coverage.

5.7 Reporting Format

We summarize results using: (i) per-crate coverage plots (TLC/TBC over time), (ii) bar charts for CHR/EHR across approaches, (iii) scatter plots for NTC vs. TLC (RQ6), and (iv) tables listing unique crashes and TTFC.

5.8 Threats to Validity

We discuss common threats: (i) benchmark representativeness (external validity), (ii) fairness of baseline implementations and harness effort (internal validity), and (iii) measurement bias in coverage instrumentation and crash deduplication (construct validity).

References